Structure of Design

Team Number: 10

Topic Number: 3 Ticket Vending Kiosk

Dominic Callueng (110167700)

Marcus Amero (110173570)

Spence Sharron (110162470)

Hassan Hajje (110142526)

Abdulrahiim Mahamud (110169194)

Ibrahim Mohamed

Introduction

This project is a software simulator for a light rail ticket vending kiosk. The simulator will be programmed with Object Oriented Programming C++ in mind. The simulator models the customers purchases, ticket/card dispensing, and maintenance operation so the transit authority can test the software's performance during peak hour. It also simulates constraints such as limited hopper and ticket/card capacity, coin only returns and hardware glitches like missed touchscreen presses and card writer jams. The software operates in sales mode or service mode and can produce statistic of the kiosk. The system is based on a strict separation of user-end transaction logic and back-end inventory management by using a modular Object-Oriented architecture. The further addition of the Qt system guarantees a better user interface/experience, which recreates the real customer experience and responds to more complex user inputs. This prototype is a software simulation that links hardware design with software simulation. This prototype creates a resemblance of the real world by connecting Arduino based hardware with a modular, object-oriented backend, allowing testing of the precision of the fares.

(Figure 1: Conceptual 3D-printed kiosk prototype. Generated by **Gemini (Google AI)**, 2026.)

Problem statement & Project Scope

Transit authority needs to verify the software simulation can successfully operate during high-demand periods will maintaining correct fare accounting and handling real world limitation.

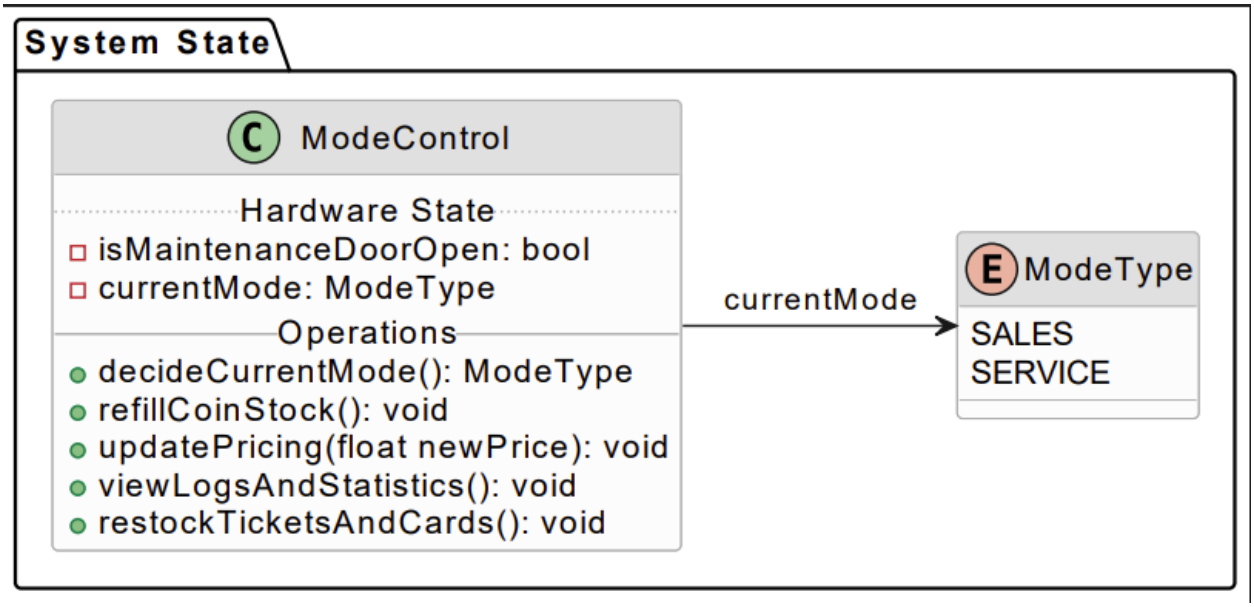
- Operate in Sales Mode: offer selection on product, payment types, change in coin denominations only, ticket or card dispensing, refunds on failure.
- Operate in Service Mode: transit authority can refill hoppers, collect money, restock tickets/cards, edit pricing, and view logs.
- Product constraint: accepts only specific denominations, limited hopper capacities, low stock warning, 3 second card encoding time, 3% touchscreen missed press, card-writer jams.

System Design

Mode Class – Spencer , Ibrahim, Dominic:

This class controls if the kiosk is operating in Sales Mode or Service Mode.

- Decides current Mode (Sales/ Service)
- Refilling coin stock
- Update pricing
- Logs and statistics
- Restock tickets/cards



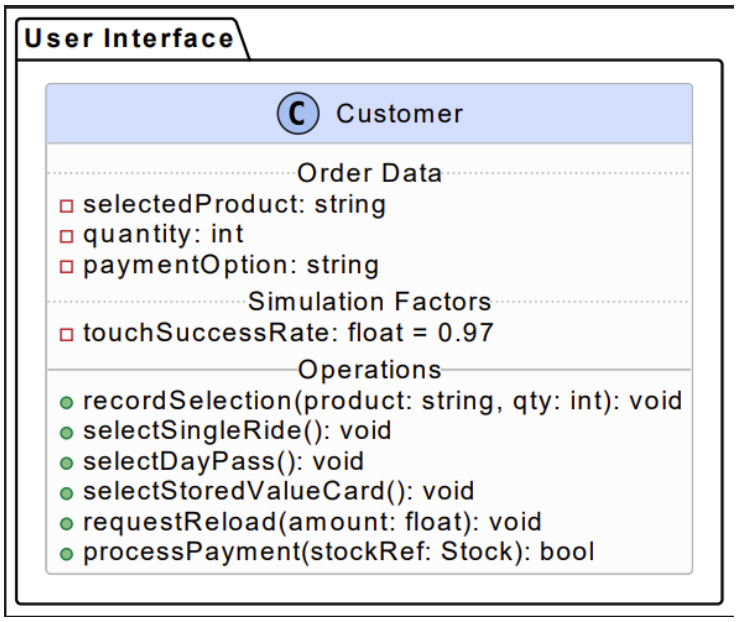
(Figure 2: ModeControl class and ModeType enumeration Class Diagram of the UML of the mode control and mode type enumeration which depicts the system state. The PlantText was used to create this diagram and contains the standardized icons of the public and private visibility.)

Customer Class – Marcus and Ibrahim, Dominic:

This class represents each customers order and payment history. It stores information like what products they want to buy and their payment option.

- Records selected product and quantities
- Single ride tickets
- Day passes
- Stored values cards
- Reloads

Customer class will cooperate with Stock class for payments.



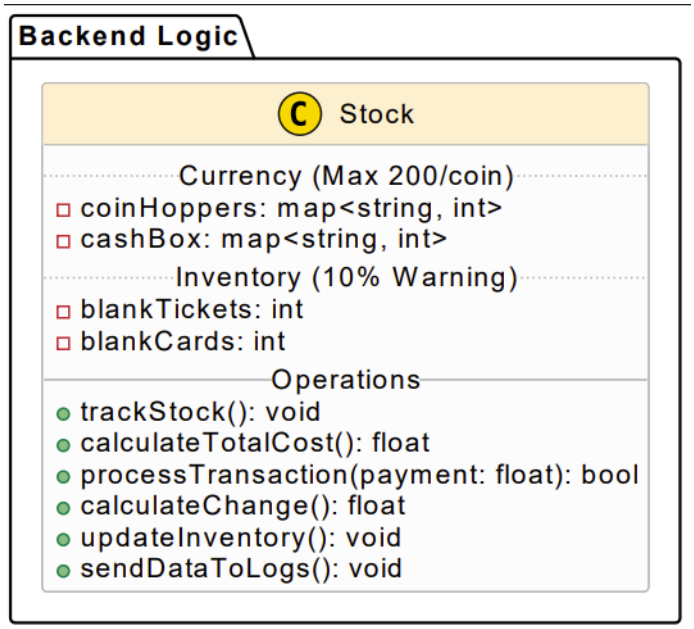
(Figure 3: UML Class Diagram of Customer class belonging to the package, User Transactions. This diagram, which was drawn with the help of PlantText, describes the attributes and operations in regards to product selection, and order information, and it is noted that it cooperates with the Stock class.)

Stock Class - Hassan and Heem, Dominic:

This class keeps track of the kiosk's internal supplies and money storage. It tracks quantities of each money denomination, ticket/ card stock, and handles to transaction logic. It controls inventory updates, payments, refunds, and send this information to logs.

- Track stock
  - Coin and cash hoppers in all denominations
  - Blank tickets/cards
- Transactions
  - Receives request from customer class.
  - accepts/ rejects cash denominations from customer's order

- calculates change in coin denominations only.
- manages stock after a transaction



(Figure 4: Stock class, UML Class Diagram, in the Backend Management package. It was written with the help of PlantText and describes the characteristics and functions of currency and inventory supply, including a specific indication of change calculation with coins only.)

GUI Class – Spencer and Hassan, Dominic (QApplication and QWidget):

This class handles the Graphics User Interface through the Qt framework (Qt widgets). This class is responsible for the graphics, buttons, widgets etc. It is also responsible for handling inputs from the user via buttons presses.

- Displays kiosk screen
  - Product selection buttons
  - Payment input
  - Confirm / cancel prompts
- Display Service Mode

- Refill buttons
- Price control buttons
- Display logs and statistics

Functional Requirements

- Sell: single ride ($3.50), day pass ($10), stored – value loads in ($10/ $20/ $50), and load existing stored-value card
- Accept payments accepts $0.25, $1, $2 coins. Accepts $5/ $10/ $20 bills only. Accepts credit/debit cards.
- Change is returned to the customer in coins only. Hoppers of each denomination has a 200 max capacity.
- Service Mode: refill all money denominations, remove cash, restock, update fare, view logs/statistics, return to Sales Mode.
- Low stock warning when capacity of ticket/card stock is below 10% of max capacity.
- Touchscreen misses 3% of the time.
- (Will add if we have enough time) Physical system to pay using credit/debit or load existing stored- value card with RFID sensor.

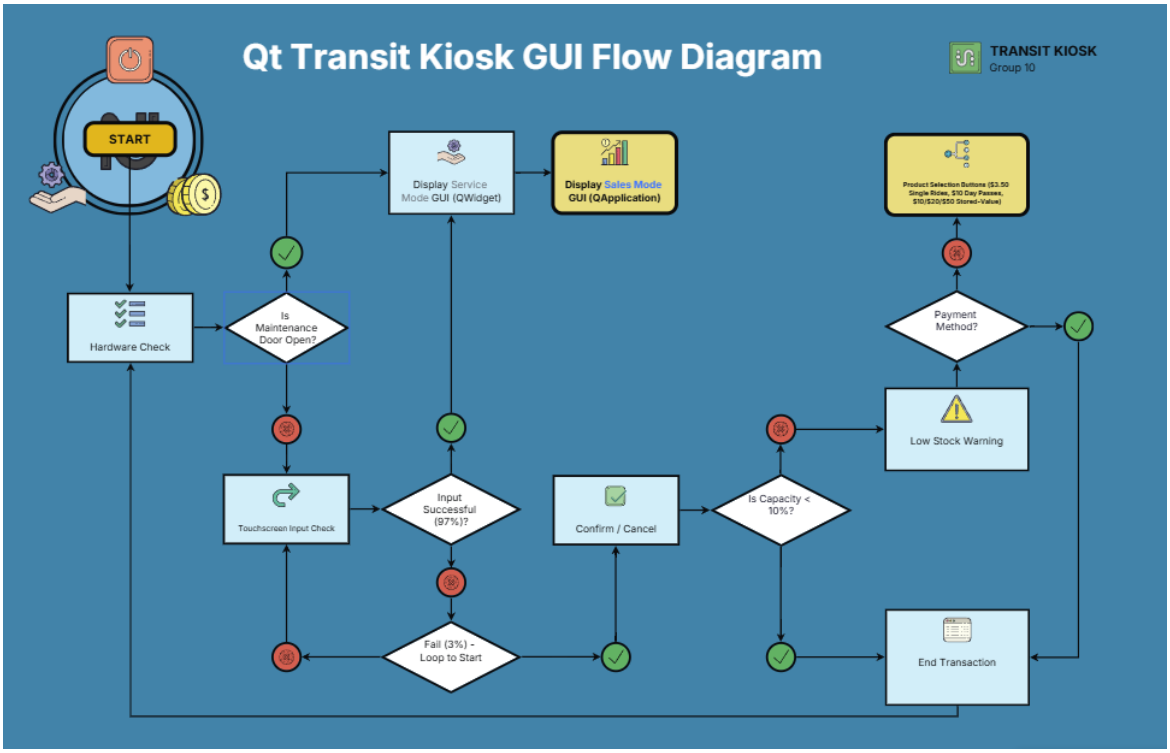
Non-Functional Requirements

- Keep fare totals, payments, refunds, and change amounts correct during all transactions.
- Run smoothly without slowing down too much, even when simulating busy/peak times.
- System should not crash when errors happen (missed press, rejected bill, not enough change, invalid inputs).

- Transaction logs and simulation statistics should be easy to read and kept in a consistent format.

- Simulation should produce repeatable results when checking performance.

(Figure 5: Horizontal flow chart of the Qt-based Transit Kiosk Sales and Service modes. Developed through



Venngage, it shows the 3 percent touchscreen error cycle, 20 bill rejection logic, and 10 percent low-stock logic.)

Algorithms

The simulator uses simple decision-making techniques to mimic how the ticket kiosk would work in real life. It considers the product chosen by the customer, makes the payment, ensures that the amount is correct, and computes the coins to be given as change based on the availability of coins in the machine. If the exact amount is not given as change, the program computes the amount that is closest to the amount owed by the machine and prompts for confirmation. In

automatic mode, random number generation is used to mimic situations that a real-life machine would experience, such as the arrival of a customer or a customer failing to touch the screen on the machine.

Libraries & Tools

Framework: Qt Widgets, Arduino

The Qt framework offers the necessary tools to create a GUI for this simulation. Arduino could be implemented for our debit/credit and stored value cards.

Language: C/C++

Libraries: QApplication, QWidget, QPushbutton, QVector, QDebug, QMap, QSerial, QRandomGenerator

QApplication is the library essential for running the GUI. QWidget is the parent class to all interactive widgets like buttons, sliders etc. QPushbutton inherits from QWidget. QVector will be used to heap allocate Customer objects. QDebug is a way to print to serial for debugging. QMap will be used for our money; the key is the value of the denomination, and the value is the quantity of the denomination. QSerial is how we can communicate our RFID Arduino based circuit with our GUI. QRandomGenerator is a library we can use to generate simulations and faults.

Task Distribution

The assignment will be divided up by the major classes/components so each member can focus on a specific part of the system. Dividing the work like this will assist us in keeping everyone's responsibilities clear and to work efficiently with less confusion from overlap. Although to

prevent any issues through the integration process of all the classes, Dominic will be working alongside and supporting each group to ensure nobody is implementing anything that would not work well with another class. The tasks will be delegated as follows:

Mode class: Spencer, Ibrahim

GUI: Spencer, Hassan

Customer class: Marcus, Ibrahim

Stock class: Hassan, Abdulrahiim

Tools: Github, ChatGPT [1] PlantText, "UML Class Diagram Generator," 2026. [Online]. Available: https://www.planttext.com/. [Accessed: Mar. 1, 2026].

[2] Venngage, "Infograph Flowchart Maker," 2026. [Online]. Available: https://infograph.venngage.com/. [Accessed: Mar. 1, 2026]